

AI BILL OF MATERIALS · V0.2

Know what is inside. Show why it needs review.

An evidence-backed inventory and policy analyzer for AI projects.
Discover components. Explain findings. Track changes.

Offline analysis

No LLM dependency

MIT licensed

AI dependencies are more than packages.

Model references, loader settings and missing provenance can be scattered across a repository.

01 / VISIBILITY

What do we depend on?

Package declarations alone do not explain which model or dataset a source file references.

02 / REVIEW

What needs attention?

Mutable model references and permissive loading settings need traceable review—not a mystery score.

03 / CHANGE

Did the update help?

A release needs a comparison showing which findings appeared, disappeared or remain unresolved.

Built for: AI developers, engineering reviewers and teams demonstrating responsible release practices.

From source files to inspectable evidence.

A local browser interface and Node.js engine. Selected source is parsed—not executed.

01 →

Select

A local project folder or bundled demo fixture.

02 →

Parse

Babel for JS/TS; Lezer for Python; supported package manifests.

03 →

Inventory

Component references, exact npm lock entries and file evidence.

04 →

Assess

Explicit rules, confidence, remediation and coverage gaps.

05

Compare

Baseline differences, observed graph, hashes and JSON export.

Runtime scanning is offline. No model inference, API key, source execution or external advisory request is required.

One workspace. Six practical capabilities.

Available in the working prototype today.

AI inventory

Supported model and dataset references alongside npm and Python declarations.

Evidence-backed findings

Rule, path, line, confidence and a recommended next action.

Observed graph

Project, files and component references—not inferred training lineage.

Release comparison

Added / removed components, introduced / resolved findings and changed files.

Input fingerprints

Per-file SHA-256 hashes and a deterministic digest of selected inputs.

CLI + report export

JSON reports, baseline scans and exit-code policy gates for CI.

Clear decisions instead of an opaque score.

FAIL**A blocking rule matched.**

Examples: an unpinned model or explicit permission to load remote code.

REVIEW**Evidence is incomplete.**

Examples: dynamic options, unknown licenses, parse errors or missing local provenance.

PASS**Supported checks passed.**

No blocking rule or review condition was found. This is not a safety guarantee.

Configuration checks

Model revision format; remote-code permissions; selected Python object-deserialization calls.

Synthetic advisory check

Exact fictional npm package/version match. Demonstrates the mechanism, not live CVE coverage.

Severity and confidence are separate. A configuration finding is not proof of exploitation.

The contribution is the evidence-to-change loop.

Not a claim to invent AIBOM. A focused combination that makes AI configuration review demonstrable.

STRUCTURED, NOT JUST TEXT

Syntax-aware discovery

Parses supported executable constructs and avoids treating comments or quoted examples as calls.

UNCERTAINTY IS VISIBLE

Honest policy decisions

Separates explicit violations from unresolved evidence instead of treating “not detected” as safe.

IMPROVEMENT IS INSPECTABLE

Reproducible comparison

Connects findings, observed references, input fingerprints and baseline changes in one report.

Why it is not a wrapper: the core work is implemented in parsers, deterministic rules, report comparison and a hand-labelled benchmark—not delegated to an LLM.

Two bundled samples. A visible before and after.

The samples are included. Buttons load them on demand; nothing in the sample source is executed.

01 / RISKY SAMPLE

- Unpinned model references.
- Remote-code permission enabled.
- Unrestricted loader configuration.
- Fictional package matches a synthetic advisory.

02 / FIXED SAMPLE

- Literal revision pins and restrictive settings.
- Fixture package version updated.
- **Six blocking findings resolved** in the demo comparison.
- Unknown licenses still require review.

Present it: Run risky sample → inspect evidence and graph → run fixed sample → show the diff → run benchmark → export JSON.

Measured against explicit expected outcomes.

Snapshot of the verified prototype—not a claim about accuracy on unseen projects.

52

Automated tests passed

Engine, inventory, benchmark, CLI, HTTP and public-hosting controls.

43 / 43

Labelled fixtures passed

Positive and negative examples, including dynamic input and malformed syntax.

40 / 0 / 0

TP / FP / FN

Matched rule + file path + line within the selected fixture rules.

Ground truth is hand-labelled. Wrong evidence locations, duplicate detections and scan failures count against results.

The 43 cases run inside the benchmark tests; they are not an additional independent validation set. Synthetic advisory data only. No real-world generalization has been established.

Useful evidence. Explicit limits.

What the prototype establishes

- Observed references in supported syntax.
- Declared package metadata and exact npm lock entries.
- Policy matches and unresolved coverage.
- Identity of selected input contents through hashes.
- Differences between same-project reports.

What it does not establish

- Real vulnerability-feed coverage or verified upstream licenses.
- Training-data lineage or trusted model origin.
- Scope-sensitive data flow or runtime reachability.
- Model fairness, hallucination or prompt-injection resistance.
- Signed attestations or SPDX / CycloneDX compliance.

Input hashes are unsigned, not provenance certificates. A finding can disappear because a file was removed or unparseable—check coverage before calling it fixed.

Render-ready. Samples and visitor uploads.

One Node web service. No database, API key or paid model service required.

BUNDLED DEMONSTRATION

Try without uploading

Run risky and fixed samples, compare findings and inspect the benchmark.
No visitor source is needed.

CONSENT-GATED UPLOADS

Inspect your own project

Public mode discloses server-side processing before upload. Source stays in application memory and is never executed.

BOUNDED PUBLIC SERVICE

Controls, not a guarantee

Host and origin checks, upload and rate limits, two concurrent scans and a 10-second worker deadline.

Privacy: never upload secrets or confidential source. Consent is not authentication; the hosting provider may retain request metadata.

Prepared and locally tested; no live deployment claimed. Render Free may sleep and cold-start. Publish the repository, then deploy its Blueprint.

Grow depth before claiming broader coverage.

A staged roadmap from hackathon prototype to more useful engineering tooling.

NEXT / ACCESS + INTEROPERABILITY

Make it easier to adopt

- Authentication and edge abuse controls.
- Standards-aligned AIBOM export.
- Python lockfile support.
- Reusable CI integration templates.

THEN / COVERAGE + EVIDENCE

Improve analysis depth

- Opt-in real advisory sources.
- Verified upstream metadata.
- Scope-aware binding and data flow.
- Independent repository evaluation.

LATER / TRUST + WORKFLOWS

Connect review to release

- Artifact provenance verification.
- Signed report attestations.
- Policy exceptions with audit trails.
- Link separate runtime evaluation results.

THE PROJECT IN ONE SENTENCE

Make AI dependencies visible, review decisions explainable, and changes measurable.

A working, offline foundation for evidence-led AI supply-chain review.

See it work: Risky sample → Fixed sample → Benchmark → Export

Local demo: <http://127.0.0.1:4317/> · Setup and scope: [project README](#)

The local URL works only on the machine running the application. No public deployment is linked.